

PRODUCT DESCRIPTION:

In this project we have used Instagram data and through various SQL Queries have reached at the solutions to various questions related to the data.

APPROACH:

I have created a database based on the data given.

TECH STACK USED:

Used MYSQL WORKBENCH.

RESULT:

Used SQL queries to reach the answers. With this project my problem solving approach in SQL has improved.

DRIVE LINK:

SQL TASKS:

A) MARKETING ANALYSIS:

1. LOYAL USER REWARD:

The marketing team wants to reward the most loyal users i.e. those who have been using the platform for the longest time.

```
select  
username,
```

```
created_at
from
users
order by
created_at
limit 5
```

Result Grid |   Filter Rows:

username	created_at
Darby_Herzog	2016-05-06 00:14:21
Emilio_Bernier52	2016-05-06 13:04:30
Elenor88	2016-05-08 01:30:41
Nicole71	2016-05-09 17:30:22
Jordyn.Jacobson2	2016-05-14 07:56:26

2. Inactive User Management:

The team wants to encourage the inactive users to start posting by sending them promotional emails. Identify users who have never posted a single photo on Instagram.

```
select u.username
from users u
left join photos p
on u.id = p.user_id
where p.user_id is null
order by u.username
```

Result Grid   Filter Rows:

	username
▶	Aniya_Hackett
	Bartholome.Bernhard
	Bethany20
	Darby_Herzog
	David.Osinski47
	Duane60
	Esmeralda.Mraz57
	Esther.Zulauf61
	Franco_Keebler64
	Hulda.Macejkovic
	Jaclyn81
	Janelle.Nikolaus81
	Jessyca_West
	Julien_Schmidt
	Kasandra_Homenick
	Leslie67
	Linnea59
	Maxwell.Halvorson
	Mckenna17
	Mike.Auer39
	Morgan.Kassulke
	Nia_Haag
	Ollie_Ledner37
	Pearl7
	Rocio33

3. Contest Winner Declaration:

The team has organized a contest where the user with the most likes on a single photo wins. Determine the winner of the contest and provide their details to the team.

```
select
    likes.Photo_id,
    users.username,
    count(likes.User_id) as like_user
from
    ig_clone1.likes likes
inner join
    ig_clone1.users users
    on photos.user_id = users.id
group by likes.photo_ID, users.username
```

```
order by Like_user desc
limit 1
```

username
Zack_Kemmer93

Hashtag Research:

A partner brand wants to know the most popular hashtags to use in their posts to reach the most people. Identify and suggest the top 5 used hashtags on the platform.

```
select
t.tag_name,
count(p.photo_id) as num_tags
from photo_tags p
inner join
tags t
on p.tag_id = t.id
group by
tag_name
order by
num_tags desc
limit 5
```

tag_name	num_tags
smile	59
beach	42
party	39
fun	38
concert	24

4. Ad Campaign Launch:

The team wants to know the best day of the week to launch ads.

Determine the day of the week when most users register on Instagram. Provide insights on when to schedule an ad campaign.

```
select
WEEKDAY(created_at) as weekday,
count(username) as num_users
from users
group by
1
```

order by
2 desc

Result Grid			Filter Rows
	weekday	num_users	
▶	3	16	
	6	16	
	4	15	
	1	14	
	0	14	

Investor Metrics:

1. User Engagement:

Investors want to know if users are still active and posting on Instagram or if they are making fewer posts.
Calculate the average number of posts per user on Instagram. Also, provide the total number of photos on Instagram divided by the total number of users.

```
with CTE as (  
  select  
    u.id as userid,  
    count(p.id) as photoid  
  from  
    ig_clone1.users u  
  left join  
    ig_clone1.photos p  
  on u.id = p.user_id  
  group by  
    u.id  
)  
select  
  sum(photoid) / count(userid) as post_per_user  
from CTE  
where photoid>0
```

Result Grid		Filter Rows:
	post_per_user	
▶	3.4730	

```
select  
  user_id,  
  count(photo_id) as num_like  
from  
  ig_clone1.likes  
group by
```

```

        user_id
order by
        num_like desc

```

2. Bots and Fake accounts:

Investors want to know if the platform is crowded with fake and dummy accounts. Identify users (potential bots) who have liked every single photo on the site, as this is not typically possible for a normal user.

```

select
    user_id,
    count(photo_id) as num_like
from
    ig_clone1.likes
group by
    user_id
order by
    num_like desc

```

Result Grid   Filter Rows		
	user_id	num_like
▶	21	257
	71	257
	5	257
	66	257
	41	257
	14	257
	57	257
	24	257
	76	257
	75	257
	54	257
	91	257
	36	257
	16	103
	96	98
	69	97
	65	86

